1. Start with the app and this guide

This PDF is the separate companion to Complaint Review. The application contains ordinary ticket assessment, batch processing, analytics and dataset inspection. Presentation instructions belong in this document.

What you will achieve

Start the project, understand its data, assess complaints, process a CSV, filter and export a review queue, inspect analytics and rehearse a clear presentation. Keep this PDF beside the browser if useful.

The supplied files

- complaint_escalation_project: Python app, saved model, data, reports, tests and documentation.

- This PDF: feature reference, operating instructions, ML explanation and presentation notes.

- Separate CSVs: the actual 3,000-ticket sample, 12-row preview, 20-ticket batch and three sample scenarios.

- docs/screenshots: current application views for offline backup.

Basic vocabulary

A ticket is one complaint. A feature is a model input. A label is the answer the model learns. A score measures how strongly an example resembles the positive label. A threshold turns that score into a routing decision.

Start in the correct folder

Extract the ZIP completely. Open a terminal inside complaint_escalation_project, where start.py and README.md are located. Do not run inside the ZIP or from the outer presentation-pack folder.

```
python start.py
```

Python 3.12 is tested. On Windows, py -3.12 start.py is an alternative. First setup creates .venv and installs pinned packages, requiring internet. Later use runs locally with the included files. Operating the app needs no API key, GitHub login or external data service.

2. Startup and navigation

Launch, step by step

- Step 1: Install Python 3.12 if needed and reopen your terminal.

- Step 2: Open the extracted project folder and run python start.py.

- Step 3: Wait for setup to finish and the dashboard URL to appear.

- Step 4: Open http://localhost:8501 or http://127.0.0.1:8501.

- Step 5: Keep the terminal running. Stop it with Ctrl+C when finished.

If port 8501 is busy, run python start.py --port 8502 and visit http://localhost:8502. The browser port must match the launched server. For installation without opening the app, use python start.py --setup-only.

Four workspaces

Workspace	Purpose
Ticket assessment	Enter one complaint and receive a recommendation. This is the default page.
Batch processing	Upload a CSV or use the sample batch, then filter and export a review queue.
Analytics	Inspect the fixed test evaluation and validation analysis.
Dataset	Inspect included records, class distribution and missing values.

Use Workspace in the sidebar to change pages. On a narrow screen, open the sidebar using the arrow at the top left.

Routing settings contains the Routing threshold slider. Reset workspace clears the assessment, returns to Ticket assessment, restores New ticket with blank text and resets the threshold to 0.12. It does not delete files or retrain the model.

The sidebar identifies the model and synthetic sample environment. The app contains no tutorial page, teaching script or PDF download. This guide is supplied separately.

3. Purpose, target and prediction time

The business use

A support team needs help prioritizing complaints. The prototype recommends Escalate for review or Standard support review. A person confirms the decision; the app does not contact staff or update a real ticket system.

The target

High/Critical priority maps to escalation_required = 1; Low/Medium maps to 0. This is a high-priority proxy for escalation. Real future escalation events have not been observed in this synthetic sample.

The prediction moment

The model is intended for first agent review, when response wait is known. An arrival-time version must remove that waiting-time input and be retrained.

Input	Meaning
complaint_text	Customer wording available at assessment time.
product	Product category, such as Laptop.
ticket_type	Request category, such as Technical issue.
subject	Categorical topic, such as Device not working.
channel	Email, Chat, Phone or Social media.
response_hours	Elapsed hours from creation to first response.
previous_contacts	Earlier contacts about this same issue.

Priority defines the label and is excluded from inputs. IDs, demographics, satisfaction, final status, raw timestamps and provenance are also excluded. Closing-time fields can reveal information unavailable at assessment time.

All supplied tickets are fictional. The experiment demonstrates the pipeline; it does not establish real customer performance, calibrated risk or operational benefit.

4. Enter and assess a new ticket

Ticket assessment opens with Load sample ticket set to New ticket and blank complaint text. Other fields have editable defaults; enter observed values rather than assuming those defaults describe your ticket.

Field	Action	Values / limits
Complaint text	Enter the complaint.	Required; at most 5,000 characters.
Product	Select the product category.	Laptop, Phone, Headphones, Smartwatch, Tablet.
Ticket type	Select request category.	Technical issue, Billing inquiry, Refund request, General inquiry.
Subject	Select the topic.	Device, payment, delivery, account access or product question.
Channel	Select submission channel.	Email, Chat, Phone, Social media.
Response wait (hours)	Enter actual elapsed wait.	Number from 0 to 720; zero means zero elapsed hours.
Previous contacts	Enter earlier contacts about this issue.	Whole number from 0 to 50.

Submit and inspect

- Step 1: Complete the seven inputs.
- Step 2: Click Assess ticket; the saved pipeline transforms and scores the inputs.
- Step 3: Read the recommendation, Priority score and Routing threshold.
- Step 4: Read the senior-review or standard-handling message.
- Step 5: Expand Submitted ticket to confirm the inputs actually scored.
- Step 6: Use Export assessment to save that input and output as CSV.

Blank text produces Complaint text is required and clears the previous result. Numeric controls enforce their ranges. A response at 02:00 after creation at 23:00 the previous day means 3 elapsed hours, not clock hour 2.

5. Samples, edits and assessment exports

Load a sample ticket

Choose a sample in Load sample ticket, then click Assess ticket. Selecting a sample alone does not submit the form.

Sample	Delivered score	At threshold 0.12
Routine question	About 2.1%	Standard support review
Work disrupted	About 98.3%	Escalate for review
Repeated billing issue	About 82.4%	Escalate for review

These values describe the unedited supplied inputs and unchanged saved model. Editing inputs or retraining can change them. Live_Demo_3_Scenarios.csv contains the exact examples.

Edit and resubmit

After editing a field, click Assess ticket again. Until submission, the displayed result refers to the previously submitted details. Submitted ticket lets you inspect that record.

Changing the sample hides its previous counterpart's result until assessment. New ticket gives a blank-text form. Reset workspace clears the state and restores the saved threshold.

Interpret the score

Priority score is the interface name for escalation_score. It is an uncalibrated estimate for the priority proxy. An 80% score does not prove an 80% chance of a real future escalation.

Export assessment

The CSV contains input features plus escalation_score, decision_threshold, predicted_escalation, recommendation and missing_input_cells. predicted_escalation is 1 when the score reaches the threshold. Missing-input counts concern numeric/category cells after duration derivation; blank text uses a separate placeholder policy.

6. Upload and process a CSV

Open Batch processing. Data source defaults to Upload CSV; initially it waits for your file.

Upload route

- Step 1: Expand CSV requirements and download the template if needed.

- Step 2: Prepare a CSV with the exact headers below.

- Step 3: Select Upload CSV and use the Ticket CSV file control.

- Step 4: Wait for the flagged count and review queue.

- Step 5: Inspect warnings, filter the queue if useful, and export it.

```
product,ticket_type,subject,channel,response_hours,previous_contacts,complaint_text
```

ticket_id is optional but useful for tracing results. Prediction needs no priority or target label. Extra columns are retained in exports; only the seven approved inputs enter the model.

Included batch

Select Sample batch. The app processes 20 separately generated fictional tickets: 14 flagged and 6 standard at 0.12. Download sample batch saves its input CSV so you can show or upload it yourself.

Timestamp alternative

created_at and first_response_at in ISO format can replace response_hours. If both duration and dates are supplied, they must agree within 0.02 hours. Negative elapsed time and waits over 720 hours are rejected.

Missing values and file limits

Empty numeric cells use training medians; empty categories use Unknown; blank text gets a placeholder in batch inference. New categories are accepted with a warning and may generalize poorly. Missing columns, invalid numbers and malformed files produce errors.

Limits are 5 MB and 10,000 rows. Column names are case-sensitive. Use the supplied batch to rehearse before adapting an external dataset.

7. Filter and export the review queue

The summary counts flags in the complete processed batch. Review queue controls which rows are displayed and exported.

Filter	Included rows	Sample at 0.12
All tickets	Every processed ticket	20
Flagged for review	predicted_escalation = 1	14
Standard handling	predicted_escalation = 0	6

Review the table

The display sorts highest score first. It includes ticket ID when supplied, Priority score, recommendation and text. Scroll inside the table if needed. Native table controls may offer search, column visibility, CSV download and fullscreen viewing.

Export the queue

Export queue saves review_queue.csv. It follows the selected filter and preserves original input order within that filter. It contains all input columns and prediction outputs, even though the displayed table uses a compact subset.

Select All tickets for the complete batch. Select Flagged for review for a senior-review queue. A filter can produce zero rows; its export still contains headers.

Added output columns

- escalation_score: shown as Priority score in the interface.

- decision_threshold: the cutoff used for that output.

- predicted_escalation: 1 for flagged, 0 for standard handling.

- recommendation: the suggested routing decision.

- missing_input_cells: missing numeric/category cells handled by imputation.

Formula-like free text is prefixed with an apostrophe in CSV exports to prevent spreadsheet formula interpretation. That changes exported text representation, not the original input scored.

Changing the routing threshold recomputes the batch and filters. Counts and export refer to the current input and threshold. The table's sorted order differs from the export's preserved input order.

8. Routing settings and workspace reset

Open Routing settings in the sidebar. Routing threshold ranges from 0.00 to 1.00; the delivered saved value is 0.12.

```
Flag for review when score >= threshold
```

A ticket with score 0.30 is flagged at threshold 0.12 and given standard handling at 0.50. Moving the slider changes the policy, not the score.

Where it applies

- The currently submitted individual assessment.

- The current batch, filters and queue export.

- Validation analysis in Analytics.

The saved test evaluation remains fixed. Moving the slider does not retrain, modify the model file, or replace reports on disk.

Controlled demonstration

Assess Routine question at 0.12. Its score is about 0.021. Change the threshold to 0.01 and observe a new routing decision with an unchanged score. Restore 0.12 before continuing.

Lower thresholds generally catch more positives and create more false alarms. Real policy selection needs validation data, error costs and staffing capacity.

Reset workspace

This button returns to Ticket assessment, clears submitted details and UI selections, restores New ticket with empty text, and restores the saved threshold. It does not delete files or retrain.

The saved threshold was selected using validation F2 with desired recall 85%. That preference is not a promise of future recall or proof of a business-optimal policy.

9. Read the Analytics summary

Analytics reports Logistic Regression's saved evaluation on 600 held-out synthetic tickets at threshold 0.12.

Metric	Value	Meaning
Recall	93.9%	Caught 186 of 198 positive tickets.
Precision	42.4%	Of 439 flagged tickets, 186 were positive.
Accuracy	55.8%	Correctly classified 335 of 600 labels.
F1	0.584	Harmonic balance of precision and recall.

Confusion table

Actual / predicted	Predicted standard	Flagged for review
Actually standard	149 true negatives	253 false alarms
Actually high priority	12 missed positives	186 caught positives

Review workload is 439/600, or 73.2%. That is substantial and may not be acceptable for a real team.

Why the dummy has higher accuracy

An always-standard baseline achieves 67% accuracy and 0% recall here. The low selected threshold catches many positives while accepting many false alarms. Neither accuracy nor recall alone chooses a useful operational policy.

What remains fixed

The app reads these results from reports/metrics.json. The slider does not change test counts. A final test set should not repeatedly choose a new policy.

Export metrics

Export metrics downloads the full JSON report: metadata, validation results, test results and dummy test results. Metadata includes data hash, seed, selected model, threshold, package versions and partition sizes.

10. Use the Analytics detail panels

Open the panels relevant to the question you are answering. All remain model-analysis features; they contain no presentation script.

Model comparison

The table shows mean five-fold training cross-validation average precision and fold standard deviation. The chart displays the same comparison. Logistic Regression scored about 0.693, Random Forest 0.648, AdaBoost 0.647, kNN 0.528 and the dummy 0.331. The highest-scoring non-dummy was selected.

Average precision assesses positive ranking across cutoffs. Compare it with the positive share. Fold standard deviation is not a confidence interval. Validation and test rows are excluded from these training folds.

Feature importance

The chart shuffles each raw input on validation data and measures the average-precision drop. Complaint text has the largest measured importance, followed by previous contacts and response wait. This is global predictive reliance, not causality or an individual explanation. Correlated inputs can share importance.

Threshold analysis

The sentence updates validation recall, precision, flagged count and missed positives at the current routing threshold. The chart is a saved validation plot; its vertical line marks the saved cutoff and is not redrawn by the slider.

Detailed evaluation

ROC-AUC is about 0.802; average precision 0.666; Brier score 0.167. Test positive share is 0.330. The approximate 95% recall interval is 89.7%-96.5%, describing sampling in this experiment rather than transfer to real customers.

ROC and precision-recall plots show frozen test ranking diagnostics. Brier score is squared error of score estimates, where lower is better. Representative real-world calibration has not been established.

11. Inspect and export the Dataset

Dataset describes the included 3,000-row synthetic labelled file. If you later train on another file, its provenance is in model metadata and the metric report; this page still describes the included sample.

Records

- Step 1: Select a Ticket ID from the first 12 sample records.

- Step 2: Read the complaint text, priority and reference label.

- Step 3: Inspect six structured fields; text is the seventh input.

- Step 4: Expand Record preview for all 12 preview rows together.

- Step 5: Export preview downloads these rows as sample_preview.csv.

The viewer uses a compact preview rather than browsing every one of the 3,000 rows. Export full dataset provides the complete file.

Distribution

The bar chart counts label 0 and label 1. Stratified splitting keeps approximately similar proportions in training, validation and test.

Data quality

The table counts missing cells per column. Missing values were deliberately introduced. Satisfaction is often absent because it is a later field and is excluded from modeling. Numeric/category imputation happens within the fitted training pipeline.

Export full dataset

The download is byte-for-byte identical to data/complaints.csv. It has 17 columns and seven model inputs. Extra fields include identifiers, timestamps, priority, label, demographics, final status, satisfaction and provenance.

The preview explains inputs and labels. It is not an independent test benchmark and must not be used to claim accuracy.

12. Understand the learning pipeline

The generator creates fictional tickets with varied wording, context, random outcomes and deliberate missing cells. Seed 42 makes it reproducible in the tested environment. Priority is assigned consistently after the noisy binary outcome is drawn.

Stage	What happens
Validate	Check features, labels, limits, timestamps and IDs. Remove exact duplicate rows; reject conflicting duplicate IDs.
Split	Stratified random 1,800 training / 600 validation / 600 test.
Numbers	Training-fold median imputation and standard scaling.
Categories	Unknown for missing cells, then one-hot encoding; new categories are ignored safely by the encoder.
Text	TF-IDF words and adjacent word pairs, up to 600 features.
Compare	Complete pipelines in five-fold training cross-validation.
Select policy	Model by training CV average precision; cutoff by validation F2.
Freeze and save	Evaluate the fixed training-fit model and cutoff on test; save evidence and artifacts.

Five candidates

Logistic Regression combines weighted features. kNN uses five neighbors. Random Forest combines 100 trees. AdaBoost combines 50 sequential weak learners. The dummy outputs the training positive share. kNN is not itself an ensemble.

Why pipelines matter

Imputation, scaling, category vocabulary and text vocabulary learn from training folds only. The joblib artifact stores the complete fitted pipeline, so the app and terminal apply the same transformations.

The selected model is not refitted on training plus validation after cutoff selection. This preserves the exact model and score scale evaluated on test. Real chronological or repeated-customer data would need time-based or group-aware evaluation.

13. Codebase and terminal tools

File	Responsibility
start.py / run.py	Environment setup and local launch.
app.py	Four workspaces and routing controls.
complaints/features.py	Input contract, validation, elapsed time and labels.
complaints/models.py	Preprocessing and classifier candidates.
complaints/evaluation.py	Metrics, F2 cutoff choice and intervals.
complaints/training.py	Splits, CV, selection, evaluation and saved evidence.
complaints/inference.py	Model loading, shared scoring and CSV export.
complaints/data.py / demo.py	Generator and optional sample tickets.
complaints/plots.py	Measured-result figures.

Commands after setup on Windows

```
.\.venv\Scripts\python.exe generate_data.py --rows 3000 --seed 42
.\.venv\Scripts\python.exe train.py
.\.venv\Scripts\python.exe predict.py --input data/batch_examples.csv --output reports/scored.csv
```

On macOS/Linux, use .venv/bin/python. Generation replaces sample files; training replaces artifacts and reports. Preserve the rehearsed copy before changing them.

To train another file, use train.py --data your_file.csv --min-recall 0.85 within the environment. It requires the feature schema and a 0/1 target or valid priority, at least 100 rows, and 20 examples per class. Contradictory priority and target labels are rejected.

Reports include partition IDs, frozen predictions, comparison, importance and data quality. Manifests record hashes and versions. Tests check validation, training-only preprocessing, prediction, filtering, reset and startup recovery.

14. Rehearse the live presentation

Keep this guide outside the application. Its interface contains ordinary workspaces and task controls.

Step	Action	Explain
1. Open	Start the app before class.	Local Python decision-support prototype.
2. Data	Dataset, choose one Ticket ID.	Seven inputs; high-priority proxy; synthetic data.
3. Routine	Ticket assessment, Routine question, Assess ticket.	About 2.1%; standard handling at 0.12.
4. Disruption	Work disrupted, then Assess ticket.	About 98.3%; senior review recommended.
5. Batch	Batch processing, Sample batch.	Twenty tickets; fourteen flagged.
6. Filter	Flagged for review in Review queue.	Fourteen-row senior-review queue.
7. Export	Export queue; optionally return to All tickets.	Filter controls rows; export preserves input order.
8. Evaluate	Analytics.	186 positives caught, 12 missed, 253 false alarms.
9. Close	Limits and next experiment.	Representative data and practical policy validation.

If time permits, demonstrate section 8's threshold change and restore 0.12. The main demonstration works without changing settings.

Opening line

"I built a Python complaint-review prototype that combines complaint text and ticket context to recommend senior review. I compared several models, separated evaluation from selection, and made single-ticket and batch workflows available in a normal application."

Closing line

"The contribution is a complete, reproducible prototype with visible inputs, outputs and evaluation. The next step is representative real data and a review policy matched to error costs and staff capacity."

15. Slides and timing

Aim for eight to ten minutes including the demonstration. You need not read every metric or open every panel.

Slide	Show	Main point
1. Problem	One complaint example	Support teams need to prioritize review.
2. Data and target	Dataset record or separate CSV	Synthetic sample; High/Critical maps to 1.
3. Method	Split and preprocessing flow	Safe inputs, full pipelines, training-only selection.
4. Application	Two assessments and batch queue	Consistent use of a saved model.
5. Results	Confusion counts and workload	High recall also creates false alarms.
6. Limits	Three short points	Synthetic data, proxy target, representative validation.

Time plan

Use 45 seconds for the problem, one minute for data, one minute for method, three minutes demonstrating, one minute on results and 45 seconds closing. Leave time for questions.

Show code when asked

Use features.py for approved inputs, models.py for preprocessing, training.py for split/selection, inference.py for shared scoring, and app.py for the interface. Explain responsibilities rather than reading every line.

Explain the low accuracy

"The low cutoff catches most positive examples but flags many standard examples. An always-standard baseline has higher accuracy here and zero recall. We report both missed tickets and workload, and would choose a real policy with validation data, costs and capacity."

Use your own words and the experiment's actual results. Do not replace them with a better-looking number from the original slides.

16. Sample files and data sources

Separate file	Role
Sample_Data_3000_Tickets.csv	Exact full labelled sample used by this experiment.
Teacher_Preview_12_Tickets.csv	First 12 rows, ID, seven inputs, priority and target.
Batch_Demo_20_Tickets.csv	Separate unlabelled batch input.
Live_Demo_3_Scenarios.csv	The three optional form samples.

The full file has 17 columns; only seven enter the model. Priority and target are not inputs. The preview is not a test benchmark, and the unlabelled batch demonstrates workflow rather than accuracy. The generator and SHA-256 data hash are documented in the project.

Kaggle: source attributed by the original slides

Customer Support Ticket Dataset, Suraj520

The verified page describes technology-product tickets with descriptions, priority, channel, response time and satisfaction. It lists a CSV and CC0 license. Download may need sign-in. We did not download that complete CSV or train this model on it. Its schema and timing fields need inspection and adaptation; they do not establish observed future-escalation labels.

CFPB: a different complaint-data research source

Official Consumer Complaint Database

CFPB offers downloadable financial-service complaints and an API. It cautions that complaints are not a statistical sample of all consumer experiences. Its domain and labels differ; it is not a ready-to-upload replacement here.

Sources were checked November 2, 2026. Use included data for the rehearsed presentation. External-data adaptation is a separate experiment; preparation details are in docs/DATA_SOURCES.md.

17. Viva and interview answers

What does it predict?

A High/Critical priority proxy for escalation. A future-event prediction needs recorded event labels and a horizon.

Why this problem?

It connects support prioritization with text/tabular ML, evaluation and a usable application.

Is the data from Kaggle?

The slides cite Kaggle. This model uses independently generated fictional tickets; the original CSV was not supplied.

Why did Logistic Regression win?

It had the highest mean training CV average precision among non-dummy candidates in this experiment.

What is leakage, and how is it prevented?

Held-out or future information influences learning or choice. We use safe inputs, split first, fit full pipelines within training folds, and select the cutoff on validation.

Why omit priority and satisfaction?

Priority defines the answer. Satisfaction may arrive after resolution and reveal future information.

Why TF-IDF and one-hot encoding?

TF-IDF converts words into numeric features. One-hot encoding represents unordered categories without inventing order.

Why high recall but low precision?

The low threshold catches more positives and creates more false alarms and review work.

Does an 80% score prove future risk?

No. Scores are uncalibrated and labels are synthetic priority proxies.

Can it score immediately on arrival?

This version needs first-response wait. An arrival-time version must remove that input and retrain.

Is it deployment-ready or proven fair?

That is unestablished. It needs representative evaluation, calibration, subgroup/error analysis and a human-reviewed pilot.

What next?

Meaningful real labels, time/customer-separated evaluation and a policy matched to staffing and costs.

18. Checklist and troubleshooting

Before presenting

- Start the extracted project on the presentation machine and rehearse once.
- Open this PDF separately; the app has no presentation walkthrough.
- Know the target, seven inputs, prediction time and data source.
- Rehearse two assessments, filtered batch export and Analytics.
- Keep threshold 0.12 for the main run and keep the terminal open.
- Save sample CSVs and current screenshots as an offline backup.
- Follow course rules for acknowledging sources and assistance.

Avoid

Do not claim real customer deployment, calibrated future risk, a guaranteed grade, or the source slides' 95% accuracy. Do not hide false alarms, retrain during the demonstration, or describe an old assessment as if it used unsubmitted edits.

Connection refused

Run python start.py in the project folder, wait for the URL and use its port. Try 127.0.0.1 instead of localhost. If busy, use --port 8502.

Interrupted setup

Retry python start.py with Python 3.12. It checks package versions and repairs missing pip when installation is needed. First installation needs internet; routine use of the included model does not.

Stale result or wrong export count

Click Assess ticket after form edits. Submitted ticket shows what was scored. Reset workspace restores defaults. Queue exports follow Review queue; select All tickets for the complete batch.

CSV error

Check exact headers, numbers, timestamp agreement and file limits. Use the supplied batch or template to isolate format problems before adapting another dataset.

Live app unavailable

Show saved views in docs/screenshots and use this PDF for the experiment. Describe them as saved captures. Explaining target, inputs, evaluation and the workload tradeoff still demonstrates understanding.